Web Penetration Testing on Food Ordering Systems: A Parameter Tampering Case Study

Group 15

Names	:	1. Azzahra Fatma Wahidah
		2. Nazilatun Natasya Ramadhani
Class	:	4CS2
Faculty	:	Ivan Firdaus S.T

CEP CCIT

FAKULTAS TEKNIK UNIVERSITAS INDONESIA

2026

Web Penetration Testing on Food Ordering Systems: A Parameter Tampering Case Study

Batch Code : 4CS2

Start Date : February 25th, 2026

End Date : March 02th, 2026

Name of Faculty : Ivan Firdaus S.T

Names of Developer :

1. Azzahra Fatma Wahidah
2. Nazilatun Natasya Ramadhani

Date of Submission: March 02th, 2026

ABSTRACT

Web-based food ordering systems process and store sensitive customer data such as names, phone numbers, and delivery addresses. Weak server-side validation mechanisms may allow attackers to manipulate transactional data during transmission. This study presents a web penetration testing case study on a simulated restaurant ordering application named ARDWD (Around D' World). The objective is to analyze a parameter tampering vulnerability caused by improper server-side validation and weak access control enforcement during the order submission process. Using an interception proxy tool, the attacker is able to modify order parameters before they reach the server. The results demonstrate that manipulated data is stored and processed without integrity verification, leading to data integrity violations and potential fraudulent order redirection. This study emphasizes the importance of implementing strict server-side validation and authorization mechanisms in web-based food ordering systems..

Depok, 02 Maret 2026

METHODOLOGY

3.1 Testing Approach

The penetration testing approach focuses on identifying vulnerabilities related to improper server-side validation and broken access control within the order submission process. The testing simulates an attacker who intercepts HTTP requests using an interception proxy tool and modifies request parameters before forwarding them to the server.

The objective of this approach is to determine whether the application validates the integrity of client-submitted data before storing it in the database.

3.2 Normal System Flow

1. Customer selects a menu item (e.g., sushi).
2. Customer fills in personal information including name, phone number, and delivery address.
3. Customer submits the order form.
4. The system processes the request and stores the order data in the database.
5. The confirmation page displays the submitted order details.

Ideally, the server should validate and verify all submitted data before saving it.

3.3 Interception-Based Attack Simulation

In this scenario, the attacker uses Burp Suite as an interception proxy to capture HTTP requests during the order submission process.

1. The victim fills out the order form and clicks submit.
2. The HTTP POST request is intercepted before reaching the server.
3. The attacker modifies specific parameters, such as `Customer_name` and `Address`.
4. The manipulated request is forwarded to the server.
5. The server stores the modified data without performing additional verification.

As a result, the order confirmation page reflects the attacker-controlled information instead of the original victim data.

SYSTEM OVERVIEW

2.1 Application Description

The tested system is a simulated restaurant ordering web application called ARDWD (Around D' World). The application provides various international food menu options and supports a delivery ordering feature similar to commercial food delivery platforms.

2.2 Core Features

1. Multi-country menu selection (e.g., sushi, pizza, etc.)
2. Delivery ordering system
3. Order detail page
4. Payment status tracking (paid / unpaid)

2.3 Stored Order Data

Each order contains the following attributes:

1. id_order
2. Customer_name
3. no_telpon
4. Address
5. Order_item
6. Payment check

VULNERABILITY SCENARIO

The vulnerability exists in the order submission process of the ARDWD web application. The system relies entirely on client-submitted form data without implementing strict server-side validation or integrity verification.

During the order process, user input is transmitted to the server through an HTTP POST request. However, the application does not verify whether the submitted data has been altered during transmission. As a result, attackers can intercept and manipulate request parameters before they reach the server.

By modifying critical fields such as the delivery address, attackers can alter transaction data without detection. Since the server processes and stores the manipulated input directly, the system fails to ensure the integrity and authenticity of order information.

This demonstrates a critical design flaw where the application trusts client-side data without enforcing proper validation mechanisms.

ATTACK SCENARIO

The attack scenario demonstrates how an attacker can manipulate transactional data in a web-based food ordering system through intercepted HTTP requests.

In the normal ordering process, a customer selects menu items, enters personal information such as name, phone number, and delivery address, and submits the order through the application interface. The system then processes the order and displays a confirmation page containing the order details.

However, during the transmission process, the HTTP request containing order information can be intercepted using a proxy tool such as Burp Suite. Because the application does not implement strict server-side validation, the attacker can modify critical parameters before the request reaches the server.

In this experiment, the attacker modifies several parameters including the customer name, delivery address, item quantity, and product price. After manipulation, the request is forwarded to the server.

Since the application processes client-submitted data without verifying its integrity, the manipulated values are accepted and stored in the system database. As a result, the order confirmation page reflects the altered information instead of the original customer input.

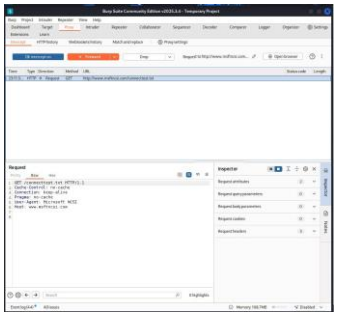
This scenario demonstrates how weak server-side validation allows attackers to alter transactional data and manipulate order information.

ATTACK ANALYSIS

5.1 Phase 1 – Request Interception

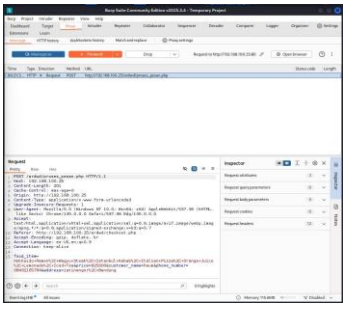
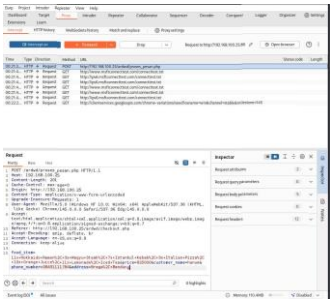
The attacker activates Intercept Mode in Burp Suite and monitors outgoing HTTP requests from the victim's browser. When the victim submits the order form, the HTTP POST request containing order information is captured before reaching the server.

Captured parameters typically include customer name, delivery address, ordered items, quantity, and price information.


Burp Intercept ON

5.2 Phase 2 – Parameter Manipulation

After intercepting the HTTP request, the attacker modifies several parameters in the request body, including the customer name, delivery address, item quantity, and product price. The manipulated request is then forwarded to the server. Because the application does not perform strict server-side validation, the modified parameters are accepted and processed as legitimate data.

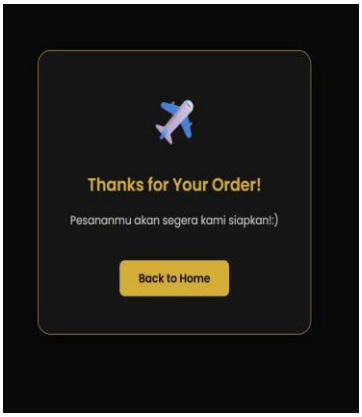
Original Parameters in Burp vs Modified Parameters in Burp

ATTACK ANALYSIS

5.3 After the modified request is processed:

- The order stored in the system contains attacker-controlled data.
- The delivery address may be redirected to a different location.
- The quantity and price of ordered items may not match the legitimate transaction value.
- The confirmation page displays manipulated order details.

This results in a **data integrity violation** where transactional data has been altered without authorization. In a real-world scenario, attackers may obtain more products than paid for or redirect orders to unauthorized locations.



Confirm order data from customer

```
mysql> SELECT * FROM orders ORDER BY id DESC LIMIT 1;
+----+---------------+-------------+--------------+-----------------------------------------------------------------------------------------------------------------------------------+----------+---------------------+
| id | customer_name | phone       | address      | food_item                                                                                                                         | price    | created_at          |
+----+---------------+-------------+--------------+-----------------------------------------------------------------------------------------------------------------------------------+----------+---------------------+
| 12 | hanun         | 08431111784 | Braga, Bandung | 11x Hokkaido Ramen, 3x Wagyu Steak, 7x Istanbul Kebab, 3x Italian Pizza, 10x Orange Juice, 11x Lemonade, Iced Tea | 825000.00 | 2026-03-13 04:24:00 |
+----+---------------+-------------+--------------+-----------------------------------------------------------------------------------------------------------------------------------+----------+---------------------+
1 row in set (0.00 sec)
```

Stored Order in Server/Admin Panel

VULNERABILITY CLASSIFICATION

6.1 Vulnerability Category

The identified vulnerability falls under the category of Broken Access Control. This category includes security flaws where applications fail to properly enforce restrictions and validation rules on user-submitted data.

In this case, the application fails to enforce server-side validation and integrity verification, allowing manipulated client-side data to be accepted and processed.

6.2 Specific Vulnerability Type: Insecure Direct Object Reference (IDOR)

The specific vulnerability identified in this study is Parameter Tampering. Parameter tampering occurs when attackers manipulate parameters exchanged between client and server in order to modify application behavior.

Because the ARDWD application does not validate or verify the integrity of submitted order parameters, attackers can alter transactional data during transmission. This results in unauthorized modification of order information and potential fraud.

SECURITY ANALYSIS

The primary security weakness in the ARDWD application is the absence of strict server-side validation and integrity verification for client-submitted data.

The system assumes that data received from the client is trustworthy. However, client-side data can be intercepted and modified using proxy tools. Without server-side validation, manipulated data is processed as legitimate input.

This issue is not related to authentication, as the victim may be properly logged in. Instead, the flaw lies in the application's failure to enforce integrity checks and validate the authenticity of submitted order parameters.

From a security perspective, this vulnerability enables data integrity violations, where attackers alter transactional data without authorization. If exploited in a real-world scenario, this could result in fraudulent order redirection and financial losses.

IMPACT ANALYSIS

The identified parameter tampering vulnerability can lead to several significant security and operational risks if exploited in a real-world environment.

1. Data integrity : Attackers can manipulate order parameters such as item quantity, price, or delivery address. This results in inaccurate transactional records stored in the system database, compromising the integrity of the application's data.
2. Financial fraud against seller : By modifying quantity and price parameters, attackers may obtain more products than the amount actually paid. This can cause direct financial losses for the seller or restaurant.
3. Order redirection: Manipulated delivery address parameters may cause orders to be sent to unauthorized locations. As a result, the legitimate customer may not receive the purchased items.
4. Operational distruption: Unexpected or abnormal orders may disrupt the seller's operations, causing confusion in order preparation, inventory management, and delivery processes.
5. Loss of customer trust: If such vulnerabilities are exploited repeatedly, customers may lose confidence in the platform's reliability and security, which can negatively impact the reputation of the service provider.

MITIGATION STRATEGIES

To prevent parameter tampering vulnerabilities, the system must implement strict server-side controls:

9.1 Server-Side Validation

All input data must be validated and sanitized on the server before processing.

9.2 Integrity Verification

The application should verify that submitted order data matches authenticated session data.

9.3 Use of Secure Communication

All communication must be transmitted over HTTPS to prevent interception.

9.4 Re-Authentication for Sensitive Changes

Modifying delivery address or transactional details should require additional verification steps.

Proper implementation of these mechanisms ensures that manipulated client-side data cannot compromise transactional integrity.

CONCLUSION

This study demonstrates that web-based food ordering systems are vulnerable to parameter tampering attacks when proper server-side validation and access control mechanisms are not implemented. Through the use of an interception proxy tool, it was shown that order submission data can be modified during transmission and processed by the server without integrity verification.

The findings reveal that the ARDWD application relies excessively on client-submitted data without performing strict validation on the server side. As a result, attackers are able to manipulate critical transactional fields such as customer name and delivery address. This vulnerability primarily impacts data integrity, potentially enabling fraudulent order redirection and operational disruptions.

Although the application may implement authentication mechanisms, the absence of server-side integrity enforcement allows attackers to exploit trust in client-side input. Therefore, implementing robust validation, authorization controls, and secure communication mechanisms is essential to protect transactional data and maintain system reliability in web-based food delivery platforms.

PROJECT DEVELOPER

Develop by	: Azzahra Fatma Wahidah & Nazilatun Natasya Ramadhani
Hardware	: HP Laptop 15-dy5xxx
Operating System	: 1. Microsoft Window 11 (victim)
	2. Kali Linux (attacker)
	3. Cybersecurity Lab VMworkstation (server)
Software	: 1. Microsoft word
	2. Microsoft PowerPoint
	3. Canva
	4. Burp Suite
	5. VirtualBox
	6. MySQL
	7. Apache
	8. PHP